



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

Google云计算原理与应用 (四)

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎



数据本身不会产生价值
只有经过分析才有可能产生价值

2.7 海量数据的交互式分析工具Dremel

- ▶ 2.7.1 产生背景
- 2.7.2 数据模型
- 2.7.3 嵌套式的列存储
- 2.7.4 查询语言与执行
- 2.7.5 性能分析
- 2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

● 产生背景

MapReduce

优点：便携

缺点：效率低

Google的团队结合其自身的实际需求，借鉴搜索引擎和并行数据库的一些技术，开发出了实时的**交互式查询系统Dremel**。



2.7 海量数据的交互式分析工具Dremel

- **Dremel支持的典型应用**

Web文档的分析

Android市场的应用安装数据的跟踪

Google产品的错误报告

Google图书的光学字符识别

欺诈信息的分析

Google地图的调试

Bigtable实例上的tablet迁移

Google分布式构建系统的测试结果分析

磁盘I/O信息的统计

Google数据中心上运行任务的资源监控

Google代码库的符号和依赖关系分析

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

▶ 2.7.2 数据模型

2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

2.7.5 性能分析

2.7.6 小结

2.7.2 数据模型

Google的数据平台常常需要满足通用性，也就是不同平台之间能够很好地实现数据的交互处理。在这种情况下想要实现上述场景中所说的实时交互查询，需要两方面的技术支撑：

首先需要有一个统一的存储平台，能够实现高效的数据存储，Dremel使用的底层数据存储平台是GFS。

另一方面还需要有一个统一的数据存储格式，这样存储的数据才可以被不同的平台所使用。

而数据存储格式的实现又包含两个方面的内容：数据模型以及模型的具体实现。首先我们来看看Dremel的数据模型。

2.7 海量数据的交互式分析工具Dremel

- 两方面的技术支撑

两方面的
技术支撑

一方面：统一的存储平台

实现高效的数据存储，Dremel使用的底层数据存储平台是GFS

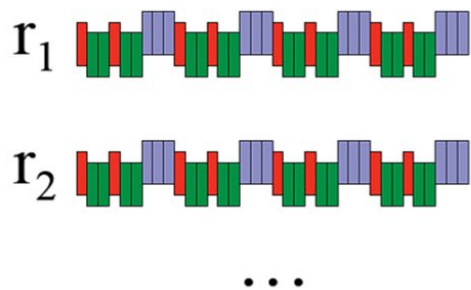
另一方面：统一的数据存储格式

存储的数据才可以被不同的平台所使用

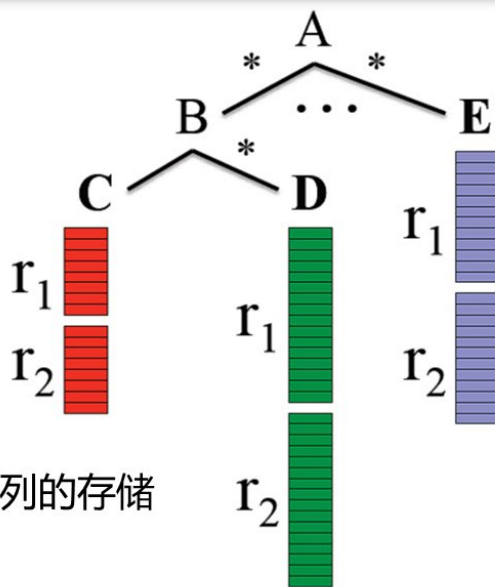
2.7 海量数据的交互式分析工具Dremel

• 面向记录和面向列的存储

Google的Dremel是第一个在嵌套数据模型基础上实现列存储的系统。



面向记录的存储



面向列的存储

好处一：

处理时只需要使用涉及的列数据

好处二：

列存储更利于数据的压缩

2.7 海量数据的交互式分析工具Dremel

- 嵌套模型的形式化定义

$$\tau = \text{dom} \mid \left\langle A_1 : \tau \left[* \mid ? \right], \square, A_n : \tau \left[* \mid ? \right] \right\rangle$$

原子类型 (Atomic Type)

原子类型允许的取值类型包括整型、浮点型、字符串等

记录类型 (Record Type)

记录类型则可以包含多个域

记录型数据包括三种类型：必须的 (Required)、可重复的 (Repeated) 以及可选的 (Optional)

2.7 海量数据的交互式分析工具Dremel

• 嵌套结构的模式和实例

```
DocId: 10      r1
Links
  Forward: 20
  Forward: 40
  Forward: 60
Name
  Language
    Code: 'en-us'
    Country: 'us'
  Language
    Code: 'en'
  Url: 'http://A'
Name
  Url: 'http://B'
Name
  Language
    Code: 'en-gb'
    Country: 'gb'
```

```
message Document {
  required int64 DocId;
  optional group Links {
    repeated int64 Backward;
    repeated int64 Forward; }
  repeated group Name {
    repeated group Language {
      required string Code;
      optional string Country; }
    optional string Url; }}
```

```
DocId: 20      r2
Links
  Backward: 10
  Backward: 30
  Forward: 80
Name
  Url: 'http://C'
```

文档的模式 (Schema) 定义

利用该数据模型，可以使用Java语言，也可以使用C++语言来处理数据，甚至可以用Java编写的MapReduce程序直接处理C++语言产生的数据集。这种跨平台的优良特性正是Google所需要的。

符合该模式的两条记录

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

► 2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

2.7.5 性能分析

2.7.6 小结

2.7.3 嵌套式的列存储

关系型数据库中采用列存储有其便利之处，因为在不同列中相同位置的数据必然属于原数据库中的同一行，因此我们可以直接将每一列的值按顺序排列下来，不用引入其他的概念，也不会丢失数据信息。

但是针对Google的这种嵌套式数据结构的列存储，数据本身之间的关系比关系数据库要复杂。存储后的数据本身反映不出任何结构上的信息，因此存储中除了记录值，还要记录结构。

另外，所有的列存储在应用时往往要涉及多个列，如何按照正确的顺序快速地进行数据重组也是列存储需要解决的。Dremel从数据结构的表示等方面解决了上述问题。

2.7 海量数据的交互式分析工具Dremel

● 数据结构的无损表示

DocId	Name.Url	Links.Forward	Links.Backward
value r d	value r d	value r d	value r d
10 0 0	http://A 0 2	20 0 2	NULL 0 1
20 0 0	http://B 1 2	40 1 2	10 0 2
	NULL 1 1	60 1 2	30 1 2
	http://C 0 2	80 0 2	

Name.Language.Code	Name.Language.Country
value r d	value r d
en-us 0 2	us 0 3
en 2 2	NULL 2 2
NULL 1 1	NULL 1 1
en-gb 1 2	gb 1 3
NULL 0 1	NULL 0 1

重复深度主要关注的是可重复类型，而定义深度同时关注可重复类型和可选类型 (optional)

每一列最终会被存储为块 (Block) 的集合，每个块包含重复深度和定义深度且包含字段值。

带有重复深度和定义深度的 r_1 与 r_2 的列存储

2.7 海量数据的交互式分析工具Dremel

● 高效的数据编码

Dremel利用图中算法创建一个树状结构

树的节点为字段的writer，它的结构与模式中的字段层级匹配。

核心的想法是只在字段writer有自己的数据时执行更新，非必要时不尝试往下传递父节点状态。

子节点writer继承父节点的深度值。

当任意值被添加时，子writer将深度值同步到父节点。

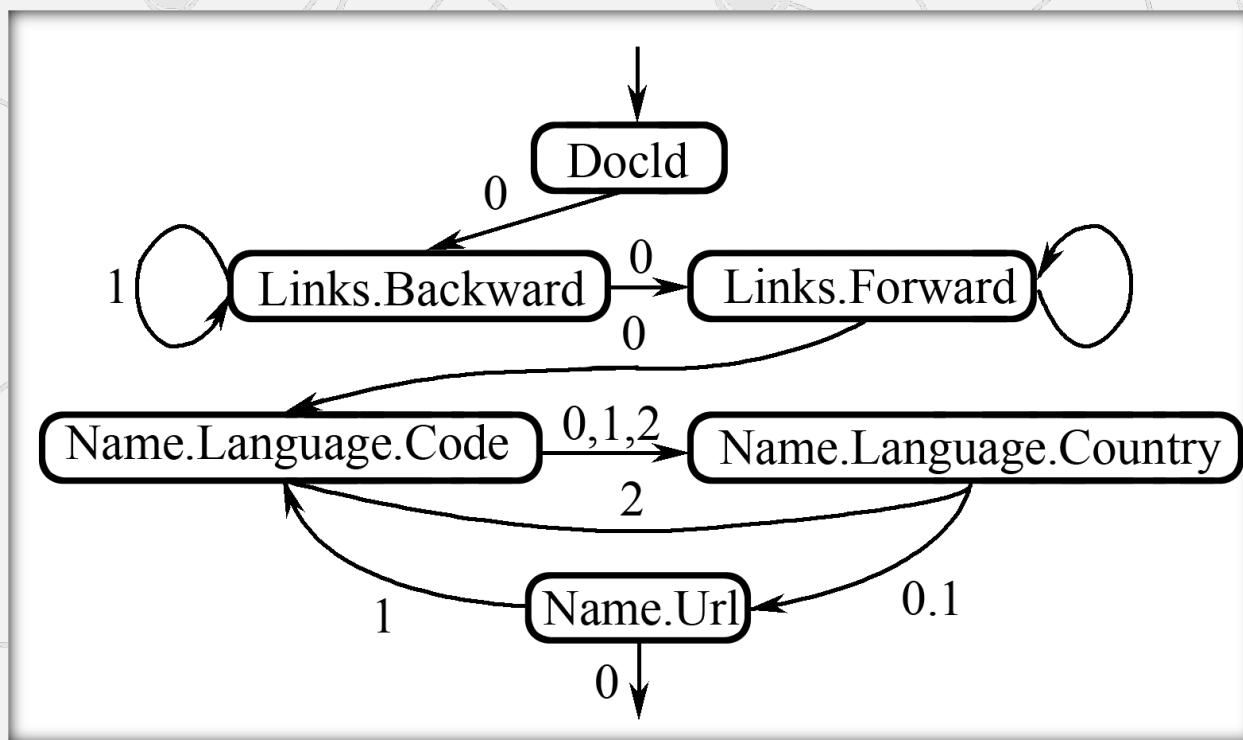
```
1 procedure DissectRecord(RecordDecoder decoder,  
2     FieldWriter writer, int repetitionLevel):  
3     Add current repetitionLevel and definition level to writer  
4     seenFields = {} // empty set of integers  
5     while decoder has more field values  
6         FieldWriter chWriter =  
7             child of writer for field read by decoder  
8         int chRepetitionLevel = repetitionLevel  
9         if set seenFields contains field ID of chWriter  
10            chRepetitionLevel = tree depth of chWriter  
11        else  
12            Add field ID of chWriter to seenFields  
13        end if  
14        if chWriter corresponds to an atomic field  
15            Write value of current field read by decoder  
16            using chWriter at chRepetitionLevel  
17        else  
18            DissectRecord(new RecordDecoder for nested record  
19                read by decoder, chWriter, chRepetitionLevel)  
20        end if  
21    end while  
22 end procedure
```

计算重复和定义深度的基础算法

2.7 海量数据的交互式分析工具Dremel

● 数据重组

Dremel数据重组方法的核心思想是为每个字段创建一个有限状态机（FSM），读取字段值和重复深度，然后顺序地将值添加到输出结果上。



2.7 海量数据的交互式分析工具Dremel

● 数据重组

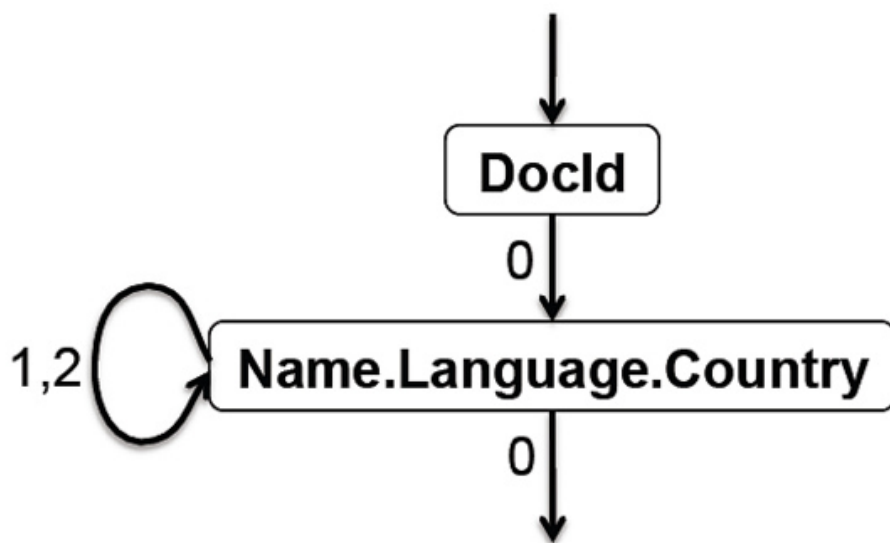
当前FSM	写入值	下一个重复深度值	动作
DocId (开始)	10	0	跳转至Links.Backward
Links.Backward	NULL	0	跳转至Links.Forward
Links.Forward	20	1	停留在Links.Forward
Links.Forward	40	1	停留在Links.Forward
Links.Forward	60	0	跳转至Name.Language.Code
Name.Language.Code	en-us	2	跳转至Name.Language.Country
Name.Language.Country	us	2	跳转至Name.Language.Code
Name.Language.Code	en	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://A	1	跳转至Name.Language.Code
Name.Language.Code	NULL	1	跳转至Name.Language.Country
Name.Language.Country	NULL	1	跳转至Name.Url
Name.Url	http://B	1	跳转至Name.Language.Code
Name.Language.Code	en-gb	0	跳转至Name.Language.Country
Name.Language.Country	gb	0	跳转至Name.Url
Name.Url	NULL	0	结束

r1的完整数据重组过程

2.7 海量数据的交互式分析工具Dremel

● 数据重组

如果具体的查询中不是涉及所有列，而是仅涉及很少的列的话，上述数据重组的过程会更加便利，下图中仅仅涉及DocId和Name.Language.Country的有限状态机。



DocId: 10 S_1

Name

Language

Country: 'us'

Language

Name

Language

Country: 'gb'

DocId: 20 S_2

Name

2.7 海量数据的交互式分析工具Dremel

● 数据重组

核心的思想如下：

设置 t 为当前字段读取器的当前值 f 所返回的下一个重复深度。

在模式树中，找到它在深度 t 的祖先，然后选择该祖先节点的第一个叶子字段 n 。

由此得到一个FSM状态变化 $(f,t) \rightarrow n$ 。

```
1 Record AssembleRecord(FieldReaders[] readers):
2   record = create a new record
3   lastReader = select the root field reader in readers
4   reader = readers[0]
5   while reader has data
6     Fetch next value from reader
7     if current value is not NULL
8       MoveToLevel (tree level of reader, reader)
9       Append reader's value to record
10    else
11      MoveToLevel (full definition level of reader, reader)
12    end if
13    reader = reader that FSM transitions to
14      when reading next repetition level from reader
15    ReturnToLevel (tree level of reader)
16  end while
17  ReturnToLevel (0)
18  End all nested records
19  return record
20 end procedure

21 MoveToLevel (int newLevel, FieldReader nextReader):
22   End nested records up to the level of the lowest common ancestor
23   of lastReader and nextReader.
24   Start nested records from the level of the lowest common ancestor
25   up to newLevel.
26   Set lastReader to the one at newLevel.
27 end procedure

28 ReturnToLevel (int newLevel) {
29   End nested records up to newLevel.
30   Set lastReader to the one at newLevel.
31 end procedure
```

有限状态机的构造算法

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

2.7.3 嵌套式的列存储

► 2.7.4 查询语言与执行

2.7.5 性能分析

2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

• 查询语言与执行

Dremel的SQL查询输入的是一个或多个嵌套结构的表以及相应的模式，而输出的结果是一个嵌套结构的表以及相应的模式。

```
SELECT DocId AS Id,  
       COUNT(Name.Language.Code) WITHIN Name AS Cnt,  
       Name.Url + ',' + Name.Language.Code AS Str  
FROM t  
WHERE REGEXP(Name.Url, '^http') AND DocId < 20;
```

```
Id: 10  
Name  
  Cnt: 2  
  Language  
    Str: 'http://A,en-us'  
    Str: 'http://A,en'  
Name  
  Cnt: 0
```

t_1

```
message QueryResult {  
  required int64 Id;  
  repeated group Name {  
    optional uint64 Cnt;  
    repeated group Language {  
      optional string Str; }  
  }  
}
```

嵌套子查询

记录内聚合

top-k

joins

自定义函数

... ..

2.7 海量数据的交互式分析工具Dremel

● 查询语言与执行

Dremel利用多层次服务树（multi-level service tree）的概念来执行查询操作

根服务器

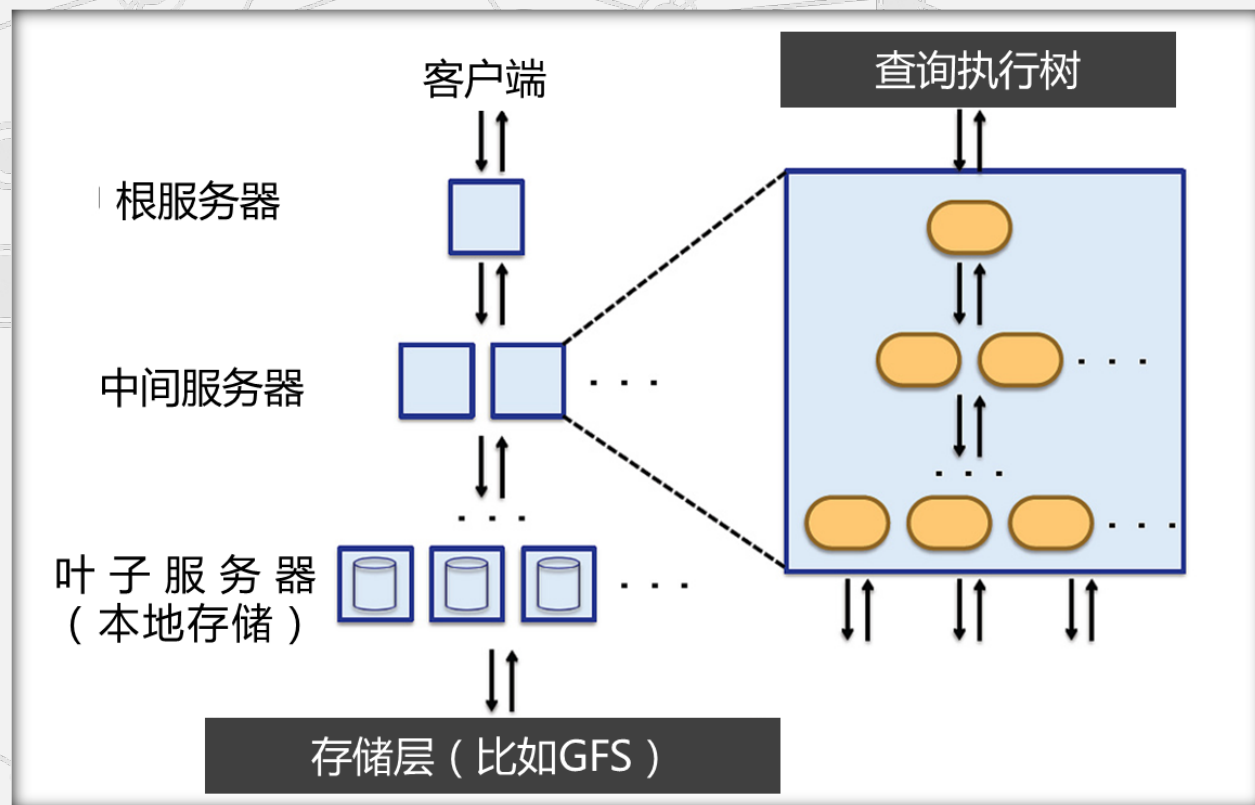
接受客户端发出的请求，读取相应的元数据，将请求转发至中间服务器。

中间服务器

负责查询中间结果的聚集

叶子服务器

负责执行数据来源



2.7 海量数据的交互式分析工具Dremel

- 查询语言与执行

Dremel中的数据都是分布式存储的，因此每一层查询涉及的数据实际都被水平划分后存储在多个服务器上。

Dremel是一个多用户系统，因此同一时刻往往会有多个用户进行查询。

查询分发器有一个很重要参数，它表示在返回结果之前一定要扫描百分之多少的tablet

2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

► 2.7.5 性能分析

2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

● 性能分析

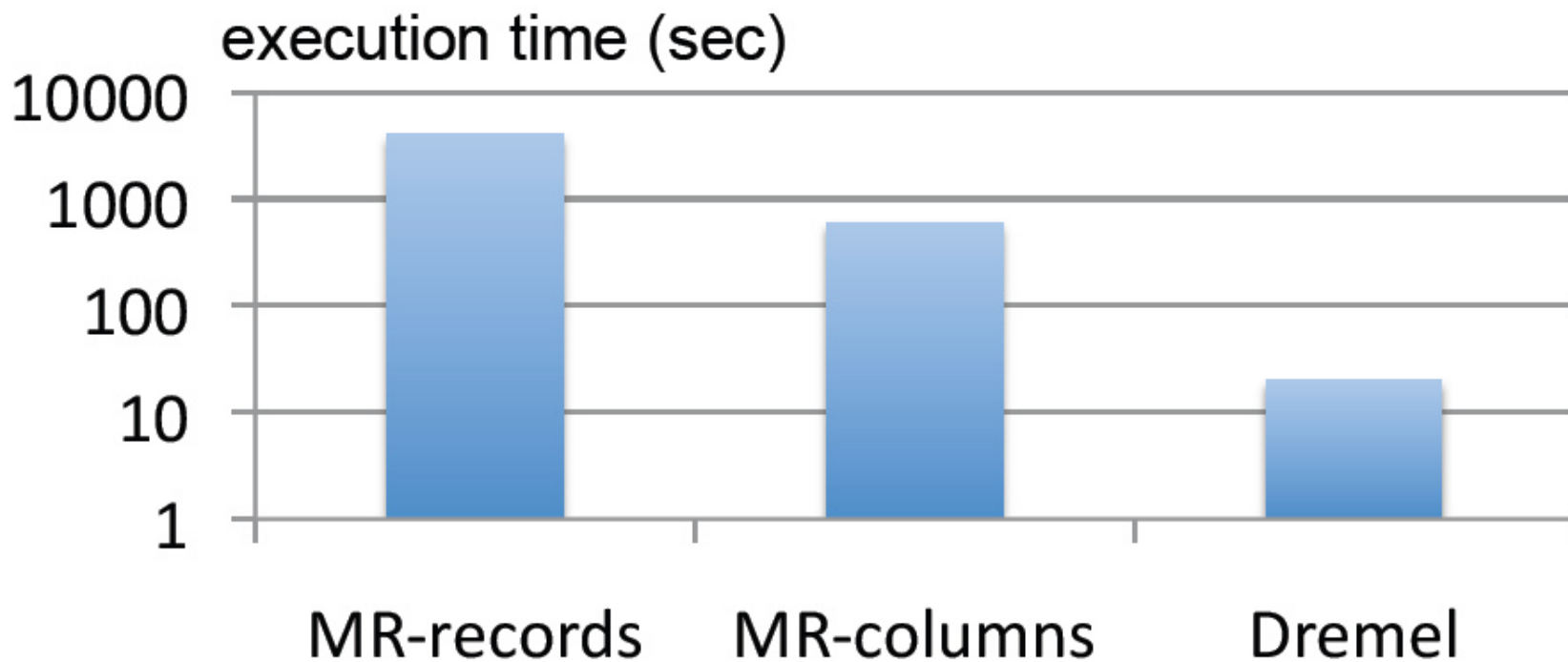
由于Dremel并不开源，我们只能通过Google论文中的分析大致了解其性能。Google的实验数据集规模如下图：

表名	记录数 (亿)	规模 (未压缩, TB)	域数目	数据中心	复制因子
T1	850	87	270	A	3
T2	240	13	530	A	3
T3	40	70	1200	A	3
T4	>10000	105	50	B	3
T5	>10000	20	50	B	2

2.7 海量数据的交互式分析工具Dremel

● 性能分析

MR从面向记录转换到列状存储后性能提升了一个数量级（从小时到分钟），而使用Dremel则又提升了一个数量级（从分钟到秒）



2.7 海量数据的交互式分析工具Dremel

2.7.1 产生背景

2.7.2 数据模型

2.7.3 嵌套式的列存储

2.7.4 查询语言与执行

2.7.5 性能分析

► 2.7.6 小结

2.7 海量数据的交互式分析工具Dremel

• 小结

1

Dremel和MapReduce并不是互相替代，而是相互补充的技术。在不同的应用场景下各有其用武之地。

2

Drill的设计目标就是复制一个开源的Dremel，但是从目前来看，该项目无论是进展还是影响力都达不到Hadoop的高度。

3

希望未来能出现一个真正有影响力的开源系统实现Dremel的主要功能并被广泛采用。

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

Dremel的推出，在一定程度上减轻了Google内部对于MapReduce系统的依赖。但是随着数据量的增加和对数据分析时时效性的要求越来越高，Dremel在有些场景下也显得不太适用，为此Google研发了内部代号为PowerDrill的系统。

该系统从2008年年底开始部署，2009年中旬开始在Google内部公开使用。到了2012年的时候，Google在 VLDB 会议上公开了Power Drill的部分实现。

根据Google自己的说法，该系统在针对相对特定的数据集上，可以比Dremel处理得更快。由于Google主要公开的是Power Drill的存储部分，因此本节主要介绍PowerDrill为了达到更快的处理速度所采用的存储方案及相关的优化等。

2.8 内存大数据分析系统PowerDrill

- ▶ 2.8.1 产生背景与设计目标
- 2.8.2 基本数据结构
- 2.8.3 性能优化
- 2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

● 产生背景与设计目标

两个假设结论

- (1) 绝大多数的查询是类似和一致的;
- (2) 存储系统中的表只有一小部分是经常被使用的, 绝大部分的表使用频率不高。

考虑两方面的内容

- (1) 如何尽可能在查询中略去不需要的数据分块;
- (2) 如何尽可能地减少数据在内存中的占用, 占用越少意味着越多的数据可以被加载进内存中处理。

PowerDrill整个系统实际分为三个部分

Web UI

一个抽象层

列式存储

2.8 内存大数据分析系统PowerDrill

2.8.1 产生背景与设计目标

▶ 2.8.2 基本数据结构

2.8.3 性能优化

2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

● 基本数据结构

下图阐述了PowerDrill采用的数据结构，简单来说就是一个双层数据字典结构。

global-dictionary dict		chunk 0			chunk 1			chunk 2		
id	search string	chunk-dict <i>ch₀.dict</i>		elements <i>ch₀.elems</i>	chunk-dict <i>ch₁.dict</i>		elements <i>ch₁.elems</i>	chunk-dict <i>ch₂.dict</i>		elements <i>ch₂.elems</i>
0	ab in den Urlaub	id	global-id	3	id	global-id	5	id	global-id	0
1	amazon									
2	cheap tickets	0	1	2	0	0	2	0	1	0
3	chaussures	1	2	0	1	1	1	1	3	2
4	cheap flights	2	4	4	2	5	4	2	5	4
5	ebay	3	5	0	3	6	3	3	10	3
6	fashingskosttime	4	12	0	4	7	0	4	10	4
7	immobilienscout			0	5	8	0			4
8	karnevalskosttime			2			1			5
9	Ia redoute			1			5			2
10	pages jaunes			3			5			1
11	voyanges snfc			2						
12	yellow pages									

全局字典表

存储全局id和搜索
关键字的对应关系

3个块的数据

块字典 记录的是块id (chunk-id) 和全局id的映射关系

块元素 记录的是块中存储数据的块id (注意不是全局id)

2.8 内存大数据分析系统PowerDrill

2.8.1 产生背景与设计目标

2.8.2 基本数据结构

► 2.8.3 性能优化

2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

● 数据分块

背景

传统的索引对于PowerDrill的查询场景作用不是很大，因此一个很自然的考虑就是对数据进行分块，过滤查询中不需要的数据块来减少数据量

方法

常见的分区方法有范围分区、散列分区等。PowerDrill实际采用的是一种组合范围分区方法。

步骤

领域专家确定若干个划分的域



利用这几个域对数据进行划分



每个块的行数达到阈值时就停止划分

局限

PowerDrill采用的数据分块方法简单实用，但是由于域的确定需要领域专家，因此这种方法在实际使用中还有一定的局限性

2.8 内存大数据分析系统PowerDrill

● 数据编码的优化

- 对于不同的块，如果我们可以确定块中不同值的数量，那么就可以根据这个**数量值**来选择可变的**比特位**来**记录块id**
- 统计一组数中不同值的个数有一个专有名词，称为“**基数估计**”
- 对于**小规模**的数据集，可以比较**容易**地统计出精确的基数。但是在**大数据**的环境下，精确的基数统计非常**耗时**，因此能保证一定精度的**基数估计**就可以满足实际的需求。
- 基数估计的方法很多，大多利用了**散列函数**的一些特性，Google内部使用的是一种称为**Hyperloglog**的基数估计方法的变种。

2.8 内存大数据分析系统PowerDrill

- 全局字典优化

优化中主要利用
两个特性

全局字典是有序的

排序后的数据常常有共同的前缀

实际使用中为了进一步减少查询中需要加载到内存的全局字典，对全局字典又进行了分块

对每个全局字典块还会维护一个布隆过滤器（bloom filter）来快速确定某个值是否在字典中。

2.8 内存大数据分析系统PowerDrill

● 压缩算法

Google曾经对一些主流的压缩算法做过简单的测试，如下图：

算法	压缩后剩余数据百分比	编码速度	解码速度
GZIP	13.4%	21 MB/s	118 MB/s
LZO	20.5%	135 MB/s	410 MB/s
Zippy/Snappy	22.2%	172 MB/s	409 MB/s

- 不管压缩算法的解压速度多快，总会消耗一定的物理资源与时间。对此PowerDrill采用了一种**冷热数据**分别对待的策略。
- 在冷热数据切换策略中，比较常用的是LRU算法。PowerDrill开发团队采用了**启发式的缓存策略**来代替原始的LRU算法。

2.8 内存大数据分析系统PowerDrill

• 行的重排

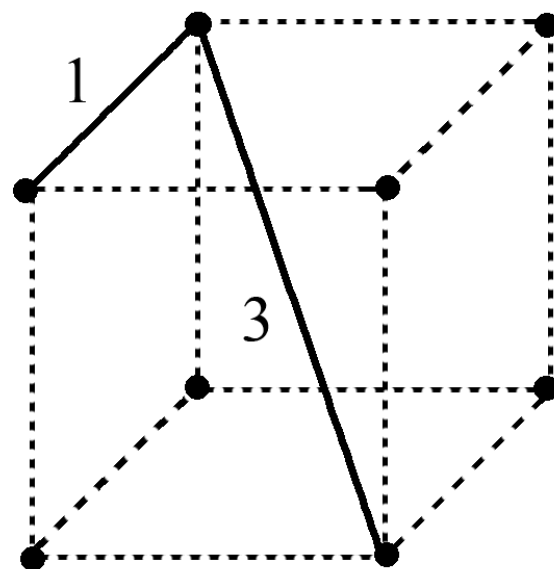
数据压缩的算法有很多，比较常用的一种称为游程编码（Run-Length Encoding, RLE），又称行程长度编码，其好处是压缩和解压缩都非常快。

数据重排的过程等效于著名的TSP（旅行商）问题

两个等长字符串之间的**汉明距离**是两个字符串对应位置的**不同字符的个数**

PowerDrill在实际生产环境中
对数据分块时选定的那几个域
按照字典序进行排序来得到重
排的结果

0	1	0
0	1	1
1	0	0



2.8 内存大数据分析系统PowerDrill

2.8.1 产生背景与设计目标

2.8.2 基本数据结构

2.8.3 性能优化

► 2.8.4 性能分析与对比

2.8 内存大数据分析系统PowerDrill

- 我们比较关注的两组数据



在查询过程中，平均**92.41%**的数据被略去

5.02%的数据会直接被缓存命中

一般仅须扫描**2.66%**的数据即可得到查询结果

超过**70%**的查询是不需要从磁盘访问任何数据的

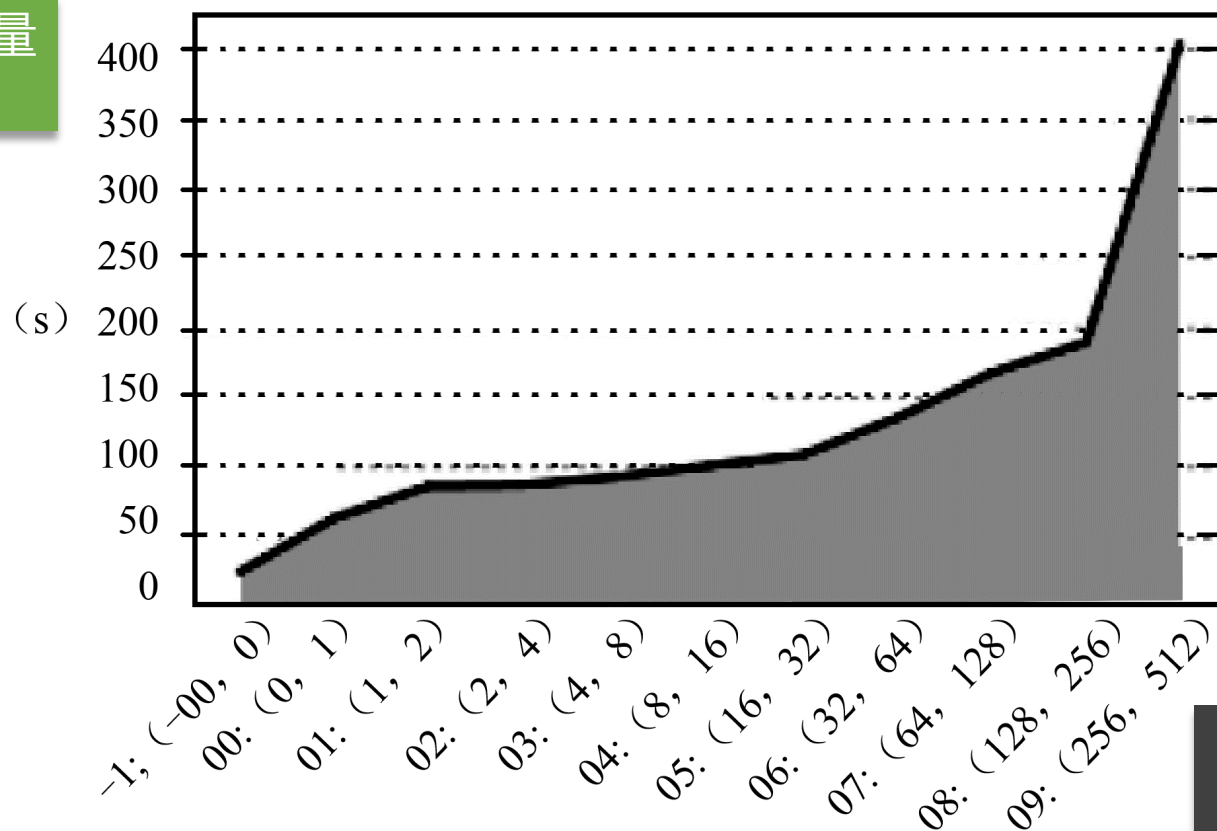
这些查询的平均访问延迟大约是**25秒**

96.5%的查询需要访问的磁盘量不超过**1GB**

2.8 内存大数据分析系统PowerDrill

性能分析与对比

访问数据量
(GB)



延迟时间
(s)

2.8 内存大数据分析系统PowerDrill

• PowerDrill与Dremel的对比

	PowerDrill	Dremel
设计目标	处理非常大量的数据集	分析少量的核心数据集
设计理念	处理的数据来自外存	处理的数据尽可能地存于内存
	未进行数据分区，分析时要扫描所有需要的列	使用了组合范围分区，分析时可以跳过很多不需要的分区
	数据通常不需要加载，增加数据很方便	数据需要加载，增加数据相对不便

2.8 内存大数据分析系统PowerDrill

● 交互式分析工具的AI进化

- > - Dremel和PowerDrill解决了海量数据的交互式查询问题，查询延迟可达秒级
- > - 今天，这一能力进化为"自然语言查询数据":
 - > - Google BigQuery + Duet AI: 用户用自然语言描述需求，AI自动生成SQL查询
 - > - ChatGPT Advanced Data Analysis: 上传数据集，通过对话完成分析和可视化
 - > - 底层仍是Dremel开创的列存储+分布式查询执行引擎

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

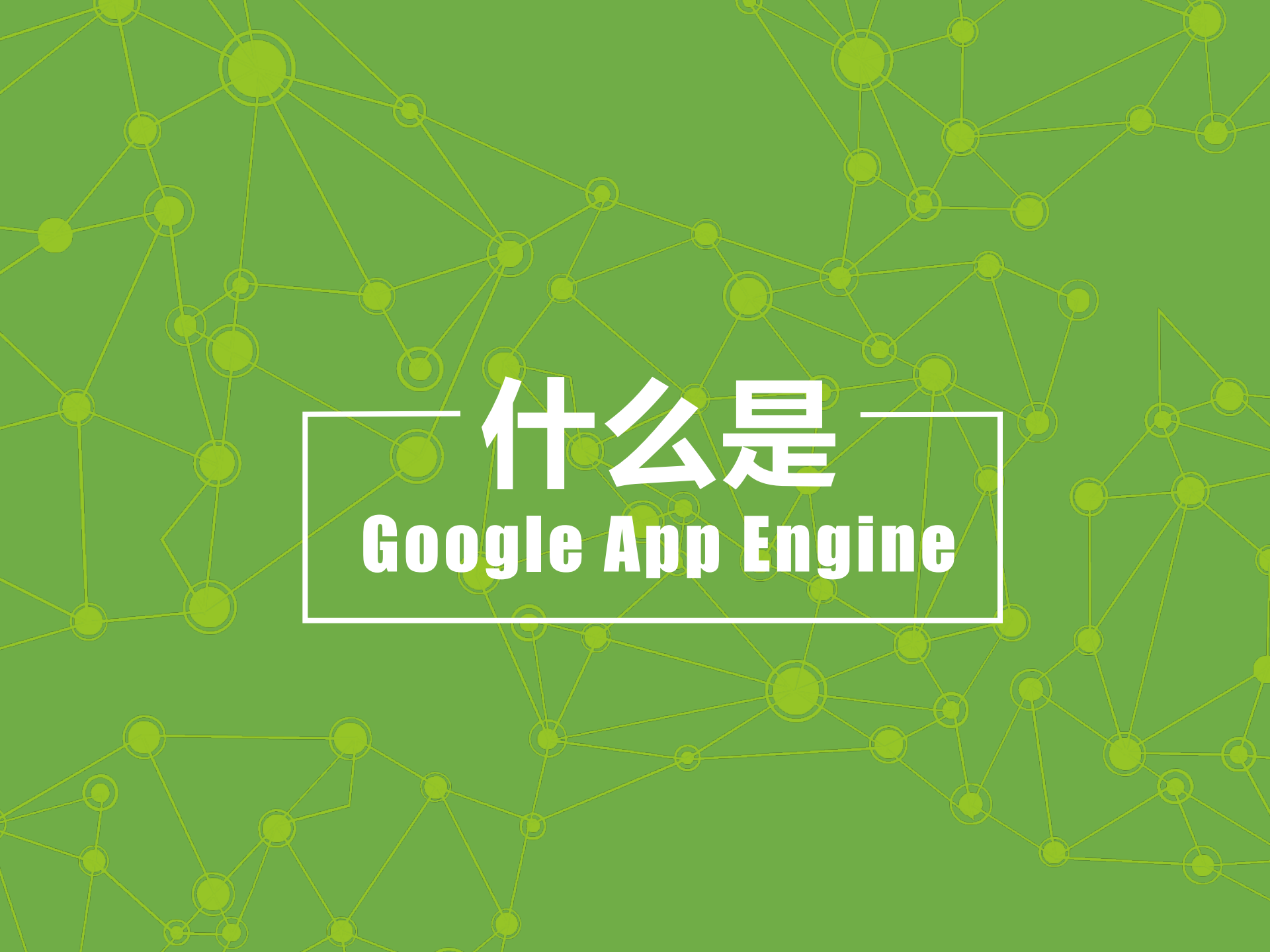
如果说Amazon给开发人员配置了一台可以在上面安装许多软件的虚拟机的话，Google App Engine可以说是给开发人员提供了一个基于Python语言的 Django框架。

由于Google App Engine与Google自身的操作环境联系比较紧密，涉及底层的操作很少，用户比较容易上手。并且Python语言相对而言简单易学，开发人员可以很容易地开发出自己的程序。

但是 Google App Engine简单方便的同时，却在提供的解决方案上有着自己的局限性。

2.9 Google应用程序引擎

- ▶ 2.9.1 Google App Engine简介
- 2.9.2 应用程序环境
- 2.9.3 Google App Engine服务



什么是 Google App Engine

Google App Engine

Google App Engine是一个由Python应用服务器群、Bigtable数据库及GFS数据存储服务组成的平台，它能为开发者提供一体化的可自动升级的在线应用服务。



Google App Engine可以让开发人员在Google的基础架构上运行网络应用程序。

在Google App Engine中，用户可以使用appspot.com域上的免费域名为应用程序提供服务，也可以使用Google企业应用套件从自己的域为它提供服务。

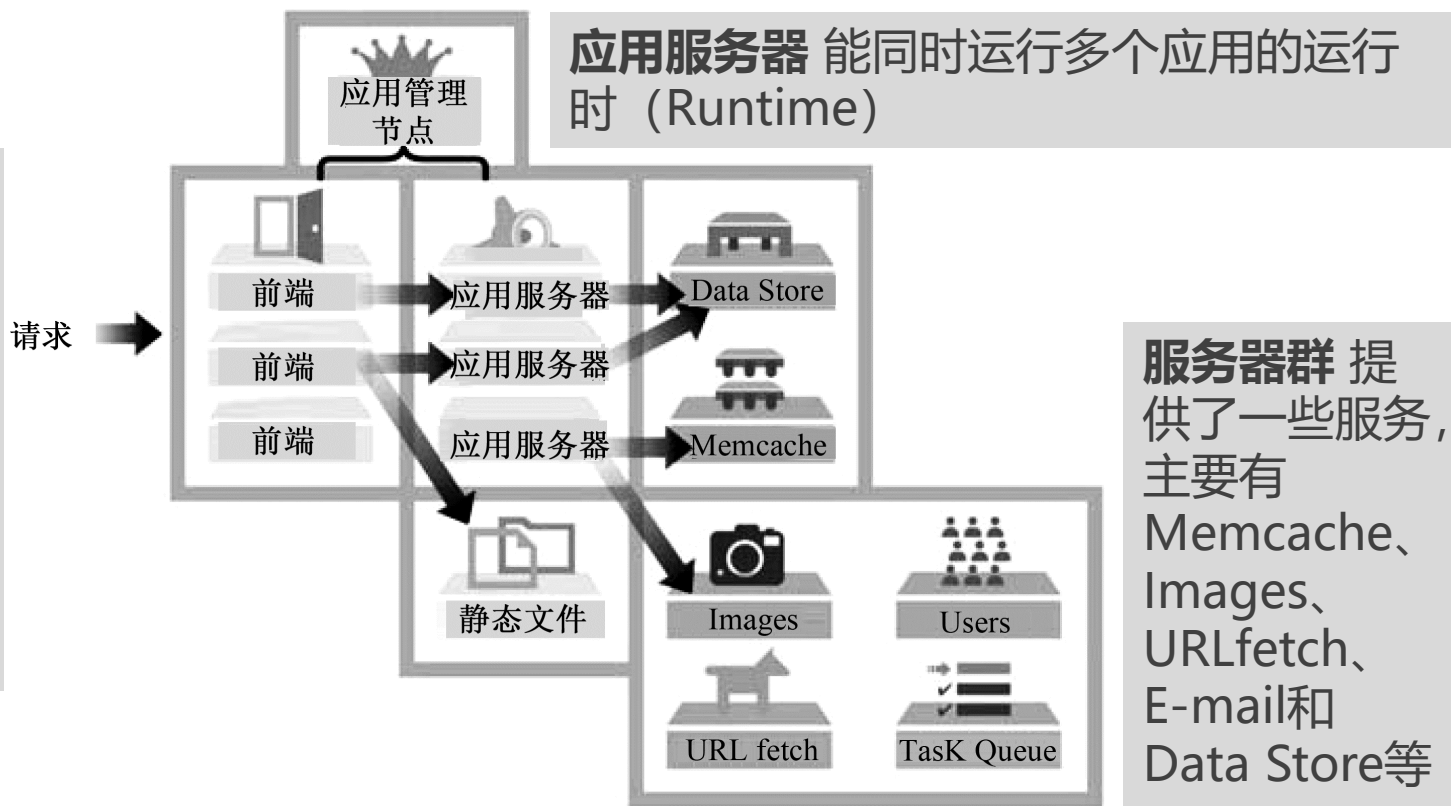
可以免费使用Google App Engine。注册一个免费账户即可开发和发布应用程序，而且不需要承担任何费用和责任。

2.9 Google应用程序引擎

• Google App Engine的整体架构

应用管理节点 主要负责应用的启停和计费。

前端和静态文件 负责将请求转发给应用服务器并进行负载均衡和静态文件的传输



2.9 Google应用程序引擎

2.9.1 Google App Engine简介

▶ 2.9.2 应用程序环境

2.9.3 Google App Engine服务

2.9 Google应用程序引擎

• 应用程序环境的特性

- 1 动态网络服务功能。能够完全支持常用的网络技术。
- 2 具有持久存储的空间。在这个空间里平台可以支持一些基本操作，如查询、分类和事务的操作。
- 3 具有自主平衡网络和系统的负载、自动进行扩展的功能。
- 4 可以对用户的身份进行验证，并且支持使用Google账户发送邮件。
- 5 有一个功能完整的本地开发环境，可以在自身的计算机上模拟Google App Engine环境。
- 6 支持在指定时间或定期触发事件的计划任务。

2.9 Google应用程序引擎

● 沙盒的限制

1

用户的应用程序只能通过Google App Engine提供的网址抓取API和电子邮件服务API来访问互联网中其他的计算机，其他计算机如请求与该应用程序相连接，只能在标准接口上通过HTTP或HTTPS进行

2

应用程序无法对Google App Engine的文件系统进行写入操作，只能读取应用程序代码上的文件，并且该应用程序必须使用Google App Engine的Data Store数据库来存储应用程序运行期间持续存在的数据

3

应用程序只有在响应网络请求时才运行，并且这个响应时间必须极短，在几秒之内必须完成。与此同时，请求处理的程序不能在自己的响应发送后产生子进程或执行代码

2.9 Google应用程序引擎

2.9.1 Google App Engine简介

2.9.2 应用程序环境

► 2.9.3 Google App Engine服务

2.9 Google应用程序引擎

• Google App Engine服务

图像操作API

开发的应用程序可以使用Google App Engine提供的图像操作API对图像进行操作，使用该API可以对JPEG和PNG格式的图像进行缩放、裁剪、旋转和翻转等操作。

1)Image类

Image类来自google.appengine.api.images模块，该类可以用来封装图像信息及转换该图像，转换时可以使用 `execute_transforms()` 方法；可以使用 `class Image (image_data)` 来构造函数，参数 `image_data` 表示字节字符串(str)格式的图像数据；可以采用PNG、JPEG、TIFF或ICO等格式对图像数据进行编码。

2)exception 类

google.appengine.api.images 包主要为用户提供了以下exception类。

(1)exception `Error()`:这是该包中所有异常的基类

(2)exception `TransformationError()`:表示尝试转换图像时发生错误。

(3)exception `BadRequestError()`:表示转换参数无效。

2.9 Google应用程序引擎

• Google App Engine服务

邮件API

Google App Engine为开发的应用用程序提供了电子邮件服务。邮件API为用户提供了两种方式来发送电子邮件，分别是 `mail.send_mail()` 函数和 `EmailMessage` 类。发送电子邮件时可以发送附件，为了安全考虑，用户发送的附件必须是所允许的文件类型。

1)允许的附件类型

允许作为电子邮件附件的MIME类型以及相对应的文件扩展名主要有:图像格式包括BMP、GIF、JPEG、JPG、JPE、PNG、TIFF、TIF、WBMP，文本格式包括CSS、CSV、HTM、HTML、TEXT、TXT、ASC、DIFF、POT，应用程序格式包括PDF、RSS。

2)EmailMessage类

邮件API中的EmailMessage类由 `google.app.engine.api.mail` 包提供。EmailMessage实例代表那些要使用Google App Engine 邮件服务来发送的电子邮件，电子邮件中有一组字段，这组字段可以使用构造函数进行初始化。

2.9 Google应用程序引擎

- Google App Engine服务

Memcache API

构造函数

实例方法

高性能的网络应用程序一般在运行之前需要使用分布式内存数据缓存(Memcache)，或用分布式内存数据缓存来代替某些任务的稳定持久存储，Google App Engine为用户提供了这样一个高性能的内存键值缓存，可以使用应用程序的实例来访问这个缓存。

Memcache适合存储永久性功能和事务性功能的数据，例如，可以将临时数据或数据库数据复制到缓存以进行高速访问。

2.9 Google应用程序引擎

• Google App Engine服务

用户API

Google App Engine的功能和账号是集成的，因此应用程序可以让用户使用他们自身的Google账户登录。

1)User对象

用户API主要是通过User类来实现其功能的，每个User类的对象代表着一个用户。User对象是唯一的且可比较，若两个对象相同，则这两个对象代表着同一个用户。

2)登录网址

用户API提供了函数来构建到Google账户的网址，这样Google账户允许用户登录或退出，并重新定向到用户的应用程序。

3)User类

User类的一个对象代表具有Google 账户的一个用户。

2.9 Google应用程序引擎

• Google App Engine服务

数据库API

Model类

Property类

Query类

GqlQuery类

Key类

Google App Engine提供了一个强大的分布式数据存储服务。该服务包含查询引擎、事务功能等功能，并且该数据库规模可以随着访问量的上升而扩大。Google App Engine数据库和传统的关系数据库不同，该数据库中的数据对象有一个类和一组属性。数据库中的查询可以检索按照属性值过滤的实体，也可以检索按照分类指定种类的实体，其中属性值可以是任何一种受系统数据库支持的属性值类型。

Google App Engine的数据库使用了简单的API来为用户提供查询引擎和事务存储服务，并且这些服务都运行在Google的可扩展结构上。在Google AppEngine中，Python接口包含了数据建模API和类似子 SQL的一种查询语言(称为GOL)。通过这些API 和GOL查询语言，可以极大地方便更用户开发可扩展数据库的应用程序。

Google App Engine的数据库 API拥有一个用于定义数据模型的机制。这里 Model 用来描述实体的类型(包括其属性的类型和配置)。数据库API提供两种查询接口：查询对象接口和GQL 查询语言。查查询的结果以Model类的实例形式返回实体，这些 Model类可以被修改并放到数据库中。

2.9 Google应用程序引擎

• 从App Engine到Google Cloud Platform

- > - Google App Engine (2008年) 是Google最早的PaaS产品, 开启了Google云计算服务的序幕
- > - 今天的Google Cloud Platform (GCP) 已发展为全球前三的公有云平台
- > - GCP核心AI/ML服务:
 - > - **Cloud TPU** (Tensor Processing Unit) : Google自研AI加速器, 专为深度学习训练和推理设计
 - > - TPU v5p (2023年) : 专为大规模LLM训练优化, 单 Pod 可达数 ExaFLOPS 算力
 - > - **Vertex AI** : 一站式机器学习平台, 覆盖数据准备、模型训练、部署和监控
 - > - **BigQuery** : 托管的数据仓库, 支持PB级数据的交互式分析

2.9 Google应用程序引擎

• Gemini——Google的大模型战略

- > - Gemini是Google于2023年底发布的原生多模态大模型系列
- > - 依托Google Cloud基础设施训练：使用TPU v4/v5p集群，算力规模全球领先
- > - 通过Google Cloud向企业提供服务：
 - > - Vertex AI Model Garden：提供Gemini Pro/Ultra等模型的API调用
 - > - Gemini与Google Workspace（Docs、Gmail、Slides）深度集成
 - > - 技术底座：GFS演进为Cloud Storage、Bigtable演进为Cloud Bigtable、MapReduce演进为Cloud Dataflow——Google核心云计算技术仍是支撑大模型的基础设施

习题：

1. Google云计算技术包括哪些内容？
2. 当前主流分布式文件系统有哪些？各有什么优缺点？
3. GFS采用了哪些容错措施来确保整个系统的可靠性？
4. MapReduce与传统的分布式程序设计相比有何优点？
5. Chubby的设计目标是什么？Paxos算法在Chubby中起什么作用？
6. 阐述Bigtable的数据模型和系统架构。
7. 分布式存储系统Megastore的核心技术是什么？

习题:

8. 大规模分布式系统的监控基础架构Dapper关键技术是什么?
9. 相比于行存储, 列存储有哪些优点?
10. 为什么MapReduce不适合实时数据处理?
11. 简单阐述Dremel如何实现数据的无损表示。
12. PowerDrill能实现高效的数据处理, 在存储部分主要依赖哪两方面的技术?
13. Google App Engine提供了哪些服务?
14. Google App Engine的沙盒对开发人员有哪些限制?



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

谢谢观看